

# Towards Graph Foundation Model: Node Feature Transfer Invariant Modeling on General Graphs

Jitao Zhao<sup>†</sup>  
zjtiao@tju.edu.cn  
Tianjin University  
Tianjin, China

Yi Wang<sup>†</sup>  
wangyi076@tju.edu.cn  
Tianjin University  
Tianjin, China

Yawen Li  
warmly0716@126.com  
Beijing University of Posts and  
Telecommunications  
Beijing, China

Dongxiao He<sup>\*</sup>  
hedongxiao@tju.edu.cn  
Tianjin University  
Tianjin, China

Di Jin  
jindi@tju.edu.cn  
Tianjin University  
Tianjin, China

Zhiyong Feng  
zyfeng@tju.edu.cn  
Tianjin University  
Tianjin, China

Weixiong Zhang  
weixiong.zhang@polyu.edu.hk  
The Hong Kong Polytechnic  
University  
Kowloon, Hong Kong

## Abstract

A significant challenge in developing graph foundational models is to achieve universality and generalizability across general graphs, beyond text-attribute graphs. Unfortunately, most graph neural networks are designed for specific applications in particular fields and are difficult to generalize across different graphs. The most critical problem is the lack of Transfer Invariant Metadata (TIM) for graphs, akin to pixels for images and vocabularies for text, which prevents the development of graph foundational models. TIM is particularly problematic, albeit critical, on graph nodes with semantically variable features, making it difficult to transfer knowledge across graphs. Here, we analyze TIM and propose a theoretical approach to mining TIM in node features. It extracts semantically consistent information across domains and unifies data space with relatively low information loss. We then introduce a Transfer-Invariant Graph (TIG) foundational model to transform features of different dimensions into a unified structural representation. This transformation can effectively learn and extract TIM and intrinsic graph knowledge by self-supervised learning. We conducted extensive experiments, and the results showed that TIG even outperformed some models trained on data from targeted domains. In one-shot cross-domain scenarios, TIG achieved high accuracy with less training data and without any prompt tuning.

## CCS Concepts

• **Computing methodologies** → **Neural networks.**

<sup>†</sup>These authors contributed equally.

<sup>\*</sup>Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License.  
WWW '26, Dubai, United Arab Emirates  
© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2307-0/2026/04  
<https://doi.org/10.1145/3774904.3792236>

## Keywords

Graph Foundation Model; Graph Pre-training; Graph Representation Learning

### ACM Reference Format:

Jitao Zhao, Yi Wang, Yawen Li, Dongxiao He, Di Jin, Zhiyong Feng, and Weixiong Zhang. 2026. Towards Graph Foundation Model: Node Feature Transfer Invariant Modeling on General Graphs. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792236>

## 1 Introduction

With the remarkable success of foundation models and Large Language Models (LLMs), Graph Foundation Models (GFMs) have recently gained significant attention [10]. Research on GFMs centers on developing universal and intelligent graph models that leverage extensive graph data to acquire highly generalizable knowledge, and can be directly applied across various data-sparse graph scenarios. Given the inherently complex nature of graph data, the exploration of GFMs is still in its early stages. Nonetheless, GFMs hold great potential to advance applications in domains such as e-commerce [26] and molecular prediction [24].

A primary challenge for GFMs is achieving universality in graph modeling, which involves learning generalizable knowledge applicable across different graphs. In Computer Vision (CV) and Natural Language Processing (NLP), foundational models benefit from semantically consistent data or shared features, such as pixels in images and vocabularies in text. This commonality, termed Transfer-Invariant Metadata (TIM), enables easy generalization. However, TIM is absent in graphs, which are manually defined abstract representations of diverse real-world objects. Graphs from different domains vary widely in structure, node features, and semantics. Although some GFMs address structural universality [17, 32], most Graph Neural Networks (GNNs) struggle to generalize when node features differ significantly in dimension and meaning. Even within

the same domain, discrepancies in node features often necessitate rebuilding and retraining models for adaptation [31]. While GFMs combined with LLMs achieve universality for text-attributed graphs, their performance remains suboptimal for general graph, significantly limiting the capabilities of graph models.

We categorize the challenge of node feature universality into two aspects: form unification and cross-domain transferability. **Form unification** involves aligning node features of varying dimensions into a consistent format to meet the fixed input dimension requirements of GNNs, while preserving semantics relevant to downstream tasks. Although essential for formal universality, the more critical aspect is **cross-domain transferability**, i.e., the unification of node feature semantic spaces across domains, similar to a shared vocabulary in NLP [13]. Node features vary widely across applications, for example, diverse word usages, molecular properties, and topological or semantic embeddings. Identifying commonalities in such semantically diverse data to create universally transferable invariant metadata is highly challenging. Without such cross-domain invariant metadata, formal unification alone cannot ensure effective model transferability across different graph datasets.

Recent efforts toward universal graph models have largely overlooked these challenges. Existing methods primarily address form unification but neglect cross-domain transferability. They often rely on data preprocessing techniques, such as Principal Component Analysis (PCA) [20, 29], or use LLMs [9] to align node features arbitrarily within a latent space of uniform dimensionality, frequently resulting in significant information loss. While LLMs handle graphs with textual features effectively, they perform suboptimally on general graphs dominated by numeric data, as evidenced by the FUG study [31]. Two notable recent works, GraphAny and FUG, demonstrate promising transferability on general graphs [31, 32]. However, GraphAny depends on labels and retraining for each test dataset, limiting its practical application, while FUG depends on data sampling and lacks robustness. Therefore, these approaches succeed only in formal feature unification, failing to guarantee transfer-invariance critical for cross-domain generalization.

To address these challenges, we investigate the hidden Transfer-Invariant Metadata (TIM) within node features. Drawing inspiration from pixels in CV and vocabularies in NLP, we propose a theory to mine latent TIM in general graph node features. This approach extends beyond traditional node encoding by leveraging relational information across different feature dimensions to extract TIM, thereby ensuring a unified embedding space. Therefore, this theory can effectively address the two previously mentioned problems to facilitate cross-domain unification of node features on general graphs. Building on this theoretical foundation, we propose a Transfer Invariant Graph foundation model (TIG). TIG considers each dimension of node features as a new node, thereby transforming raw features into transfer-invariant structures and edge weights. Additionally, dimensional augmentation is applied to these new nodes to capture transfer-invariant dimension semantics. TIG integrates an advanced graph pre-training technique [5] to self-supervise the learning of broadly generalizable graph patterns.

In summary, we make the following contributions in this paper:

- **Theory.** We propose a novel theory for extracting TIM from node features in general graphs. To our knowledge, this is the

first effort to derive such metadata from highly diverse node features, providing crucial guidance for the advancement of GFMs. Furthermore, the TIM theory holds potential for extension to broader data types beyond graphs.

- **Method.** We introduce a transfer-invariant graph foundational model that unifies the node feature space with relatively low information loss while mining TIM. This enables self-supervised learning to acquire cross-domain patterns.
- **Experiment.** We conduct extensive experiments to validate the effectiveness of our theory and model. In cross-domain settings, TIG even outperforms some in-domain models and achieves superior performance without prompt tuning and with reduced training data.

## 2 Related Work

**Graph Neural Networks (GNNs).** GNNs are one of the most powerful methods for graph representation learning [34]. They typically follow the paradigm  $Z = \sigma(\hat{A}XW)$ , which simultaneously encodes both structural and feature information of the graph. Here,  $\hat{A}$  is the propagation matrix,  $X$  is the node feature matrix,  $W$  represents learnable parameters, and  $\sigma$  is the activation function. GCN [7] is a classic and powerful GNN that directly aggregates information from neighboring nodes. GAT [22] computes attention weights between each pair of nodes to adaptively aggregate topological information. However, these methods are often constrained by the fixed input dimensionality requirements of the neural network parameters  $W$ , meaning they are typically designed for small, domain-specific models. To adapt to a new dataset, a new model must be rebuilt and trained, leading to high computational cost and poor scalability.

**Graph Self-Supervised Learning.** Training GNNs requires a large amount of labeled data. Graph self-supervised learning aims to mine latent information from data as self-supervised samples to train models, thereby reducing dependence on labeled data. Self-supervised learning in graphs primarily includes generative methods based on auto-encoders and contrastive pretraining approaches. Generative methods encode the original graph into representations and then reconstruct the graph structure for training, such as GAE and VGAE [6]. Contrastive methods train the GNNs by identifying positive and negative samples in the data, ensuring that nodes with similar semantics have close representations, while those with different semantics are uniformly distributed on a hyper-sphere. GRACE [36] treats different augmentations of the same node as positive samples and different nodes as negative samples. BGRL [21] trains solely by predicting the bootstrap output of positive samples. However, most existing graph self-supervised learning methods still suffer from the inherent limitations of GNNs, making it challenging for them to generalize effectively to new datasets.

**Universal Graph Foundational Models.** With the remarkable generalization capabilities demonstrated by large language models (LLMs) [33], researchers in the graph domain have increasingly explored universal graph models that can generalize across arbitrary datasets and downstream tasks [12]. Universal graph models typically follow a pre-training and fine-tuning framework. One of the earliest works to achieve cross-dataset generalization was GCC [14], which primarily focused on structural encoding but was limited to graphs without node features. More recent efforts can be

categorized into two main approaches: LLM-integrated methods and non-LLM-based methods. LLM-integrated methods leverage LLMs to encode graph features or structures, enabling cross-dataset generalization such as OFA [9]. However, these methods are inherently constrained by LLMs, making them effective for graphs with textual features, while ineffective for graphs with numerical features. Non-LLM-based methods arbitrarily unify data dimensions through techniques such as Principal Component Analysis (PCA), Singular Value Decomposition (SVD), or similarity embeddings, as seen in AnyGraph [27] and GraphControl [35]. However, these approaches often lose valuable information in data preprocessing and fail to preserve cross-domain semantic consistency. Recently, advanced methods such as FUG [31] and GraphAny [32] have been proposed to accommodate diverse graph data. FUG introduces the idea of dimensional encoding, where each feature distribution is encoded from a set of node samples to generate universal representations. GraphAny employs multiple linearGNNs with a topology selector, thereby enabling applicability to both homophily and heterophily structures. Very recently an awesome GFM, STAGE [16], calculate the statistical dependencies between features to unify the graph features. While, they lack exploration of TIM and also face the problem of information loss.

### 3 Preliminary

**General Graph.** Given a general graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ , where  $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$  represents the set of nodes, and  $\mathcal{E} \in \mathcal{V} \times \mathcal{V}$  represents the set of edges. The graph has a numerical node feature matrix  $X \in \mathbb{R}^{n \times d}$  and an adjacency matrix  $A$ , where  $a_{ij} = 1$  if  $e_{ij} \in \mathcal{E}$ , or 0 otherwise. A collection of various general graph datasets  $\mathcal{G}$  forms a dataset set  $\mathcal{D}_{\text{GG}} = \{\mathcal{G}^{(1)}, \mathcal{G}^{(2)}, \dots, \mathcal{G}^{(n)}\}$ .

**Graph Foundation Model.** Derived from the concept of foundational models in CV and NLP, a graph foundation model refers to a model that can be trained on a broad range of graph datasets and applied to a variety of downstream scenarios [10]. Given a graph foundation model  $f(\cdot)$ , it is trained using a general supervised or self-supervised loss, i.e.,  $\min \sum_{\mathcal{G}^{(i)} \in \mathcal{D}_{\text{GG}}^{\text{train}}} L(f(\mathcal{G}^{(i)}))$ , and for any new dataset  $\mathcal{G}^{(k)} \in \mathcal{D}_{\text{GG}}^{\text{test}}$ , the model can be directly adapted to downstream tasks  $\text{DownTask} = p(f(\mathcal{G}^{(k)}))$ , where  $p(\cdot)$  is a fine-tuning or prompt strategy for the downstream task.

### 4 Transfer-Invariant Metadata in Graphs

We now analyze Transfer-Invariant Metadata (TIM) to understand how it can be effectively extracted from general graphs.

**Brief Overview.** From the fields of CV and NLP, we deduce that extracting TIM requires using the same extraction method ( $\text{Ext}(\cdot)$ ) to derive the same physical properties ( $\text{Asp}(\cdot)$ ) of any object ( $R$ ). Classical neural networks encode all features ( $x_{i,:}$ ) for a node ( $v_i$ ), but these node features in different graphs often do not satisfy TIM. TIM theory proposes transforming objects from individual nodes to the set of all features ( $\mathcal{A}^{(k)}$ ) within a dataset ( $\text{DS}^{(k)}$ ), and using a consistent extraction method ( $\text{Ext}_{\mathcal{A}^{\text{Raw}}}(\cdot)$ ) to embed the same physical property, which is the relationship among elements within this set, thereby acquiring augmented TIM attributes for each feature. Ultimately, these feature attributes are propagated back to the node to obtain the node's TIM embeddings.

#### 4.1 Transfer-Invariant Metadata

**Definition 4.1 (TIM).** Given a set of datasets  $\mathcal{D}$ :

$$\begin{aligned} \mathcal{D} &= \{\text{DS}^{(1)}, \text{DS}^{(2)}, \dots, \text{DS}^{(N_{\text{DS}})}\}, \\ \text{DS}^{(i)} &= \{\text{Data}_1^{(i)}, \text{Data}_2^{(i)}, \dots, \text{Data}_{N_{\text{Data}^{(i)}}}^{(i)}\}, \\ \text{Data}_j^{(i)} &= \{\text{Ext}_1(\text{Asp}_1(R_j^{(i)})), \dots, \text{Ext}_{N_A}(\text{Asp}_{N_A}(R_j^{(i)}))\}, \end{aligned} \quad (1)$$

where  $\text{DS}$  is the datasets, and sample  $\text{Data}_j^{(i)}$  is a representation of real-world objects  $R_j^{(i)}$ ,  $\text{Asp}_k(R_j^{(i)})$  denotes the  $k$ -th physical property of  $R_j^{(i)}$ , and  $\text{Ext}_k(\cdot)$  represents the mathematical extraction method applied to that property. If there exists a fixed  $\text{Ext}_t(\cdot)$  and  $\text{Asp}_t(\cdot)$  satisfying:

$$\begin{aligned} \forall \text{DS}^{(i)} \in \mathcal{D}, \text{Data}_j^{(i)} \in \text{DS}^{(i)}, \exists M_j^{(i)}, \\ \text{s.t. } M_j^{(i)} = \text{Ext}_t(\text{Asp}_t(R_j^{(i)})) \in \text{Data}_j^{(i)}, \end{aligned} \quad (2)$$

we then say that  $\mathcal{D}$  contains transfer-invariant metadata  $M$  with respect to  $R$ .

We use  $\text{Asp}$ ,  $\text{Ext}$ ,  $R$  and  $M$  without subscripts to represent their corresponding sets. Definition 4.1 essentially states that TIM represents data extracted with the same method  $\text{Ext}_t(\cdot)$  and from the same physical property  $\text{Asp}_t(\cdot)$  across datasets. To facilitate understanding, consider an example from CV, where pixels serve as TIM. For an image set  $R_{\text{CV}}$ ,  $\text{Asp}_{\text{CV}}(\cdot)$  refers to the optical property of objects and  $\text{Ext}_{\text{CV}}(\cdot)$  converts this property to a numerical matrix using RGB channels. Since both  $\text{Asp}_{\text{CV}}(\cdot)$  and  $\text{Ext}_{\text{CV}}(\cdot)$  are consistent across datasets, RGB-based pixels are TIM. The invariance of  $\text{Asp}_t(\cdot)$  ensures data semantics are consistent, while the invariance of  $\text{Ext}_t(\cdot)$  ensures the semantics are encoded in a uniform mathematical form. Obviously, there is no TIM in the raw node features in the set of general graphs  $\mathcal{D}_{\text{GG}}$  with respect to the nodes  $\mathcal{V}$ .

**Corollary 4.2 (Nesting Property of TIM).** Consider the setting where  $M_j^{(i)} = \text{Ext}_a(\text{Asp}_a(R_j^{(i)}))$  is TIM, if there exists  $\widehat{M}$  along with invariant  $\text{Ext}_b(\cdot)$  satisfying:

$$\begin{aligned} \forall \text{DS}^{(i)} \in \mathcal{D}, \text{Data}_j^{(i)} \in \text{DS}^{(i)}, \exists \widehat{M}_j^{(i)}, \\ \text{s.t. } \widehat{M}_j^{(i)} = \text{Ext}_b(M_j^{(i)}) \in \text{Data}_j^{(i)}, \end{aligned} \quad (3)$$

then  $\widehat{M}$  is also TIM. Conversely, if  $M$  is not TIM, then  $\widehat{M}$  cannot be TIM.

**Brief proof of Corollary 4.2.** Let  $\text{Asp}_b(\cdot) = \text{Ext}_a(\text{Asp}_a(\cdot))$ . If  $M$  satisfies TIM, then  $\text{Asp}_a(\cdot)$  and  $\text{Ext}_a(\cdot)$  are invariant across  $\mathcal{D}$ , implying  $\text{Asp}_b(\cdot)$  is also invariant. Consequently,  $\widehat{M} = \text{Ext}_b(\text{Asp}_b(\cdot))$  satisfies the TIM definition. Conversely, if  $\text{Asp}_b(\cdot)$  is not invariant,  $\widehat{M}$  cannot be TIM.

**Discussion 1.** TIM focuses solely on the cross-domain invariance of data, without providing any guarantee regarding its reliability or sufficiency for downstream tasks. Specifically, when data satisfy the TIM property, it means that the data encapsulate the same semantics across different domains. However, this does not imply that it will provide sufficient information. In practice, it is impossible to enumerate all possible  $\text{Asp}(\cdot)$ . Each extraction of features from  $R$  inevitably results in some information loss. Therefore, extracting TIM must consider how well the extracted data aligns with the requirements of the downstream tasks.

**Discussion 2.** Existing methods are ineffective at mining TIM. (1) Research [2] on out-of-distribution generalization in graphs is unrelated to the focus of this paper and is unable to extract TIM from node features. These approaches typically involve learning a specific domain adaptation mapping,  $\text{Proj}(\cdot)$ , for two different datasets, A and B, to satisfy:

$$\text{Minimize DomDif} \left[ \text{Proj}_A(\text{Ext}_A(\text{Asp}_A(R_A))), \text{Proj}_B(\text{Ext}_B(\text{Asp}_B(R_B))) \right], \quad (4)$$

here,  $\text{DomDif}(\cdot, \cdot)$  measures the distributional differences between the two domains. These methods focus on generalizing knowledge between two specific domains, while we aim to improve the data generalization across all general graphs. (2) The latest works, such as FUG [31] and GraphControl [35] introduce techniques based on dimensional encoding and inter-node similarity encoding to address the transferability of features. However, FUG depends on data sampling and therefore cannot preserve complete dataset information, while GraphControl maps high-dimensional features into a single-dimensional connection weight. Both approaches may lead to substantial information loss.

**Discussion 3.** Definition 4.1 means that  $\text{Ext}(\cdot)$  and  $\text{Asp}(\cdot)$  must remain invariant across domains, whereas  $R$  may vary. This follows common knowledge about TIM in CV, where an image could be an animal, a building, or a person. This suggests that the key to discovering TIM lies in identifying an appropriate  $R$ . Moreover, as highlighted in Discussion 1,  $R$  and  $\text{Asp}(\cdot)$  must provide sufficient information to support diverse tasks.

## 4.2 Mining TIM in Features with Low Information Loss

We now consider how to mine TIM from node features. As discussed in Section 4.1, the key to finding TIM lies in identifying domain-invariant  $\text{Ext}$ ,  $\text{Asp}$ , and appropriate real-world objects  $R$ . In addition, as noted in Section 1, when mining TIM, it is essential to ensure that it maintains a unified form to fit the fixed input of neural networks, while not losing information in features. We define the original graph node features as  $x_{ij} = \text{Ext}_j^{\text{Raw}}(\text{Asp}_j^{\text{Raw}}(R_i^{\text{Raw}}))$ , where  $R_i^{\text{Raw}}$  corresponds to node  $v_i$ .

**Reconstruction of  $R$ .** From Definition 4.1,  $\text{Asp}^{\text{Raw}}$  is the artificial selection of the physical properties of  $R$ , which can vary significantly across different graph datasets. Without prior knowledge, it is impossible to understand the meaning of  $\text{Asp}^{\text{Raw}}$ . Therefore, transforming  $\text{Asp}^{\text{Raw}}$  into part of the entity  $R$ , could help eliminate this variability. The subsequent TIM mining theory is centered around three reconstructions of  $R$ . To facilitate understanding, we briefly introduce the reconstructions here: 1) Treat node  $v_i$  and  $\text{Asp}_j^{\text{Raw}}$  as one object, forming  $R^1$ . 2) Consider all  $\text{Asp}_j^{\text{Raw}}$  within a graph as one object, forming  $R^2$ . 3) Take node  $v_i$  and all its related  $\text{Asp}_j^{\text{Raw}}$  as one object, forming  $R^3$ .

$R^1$  is formally constructed as follows:

$$R^1 = \{(v_i, \text{Asp}_j^{\text{Raw}}) | v_i \in R^{\text{Raw}}, \text{Asp}_j^{\text{Raw}} \in \text{Asp}^{\text{Raw}}\}, \\ x_{ij} = \text{Ext}_j^{\text{Raw}}(\text{Asp}_j^{\text{Raw}}(v_i)) = \text{Ext}_j^{\text{Raw}}(R_{ij}^1) = \text{Ext}_j^{\text{Raw}}(v_i, \text{Asp}_j^{\text{Raw}}), \quad (5)$$

where the meaning of  $x_{ij}$  changes to represent the relationship between the node  $v_i$  and  $\text{Asp}_j^{\text{Raw}}$ , rather than the  $j$ -th feature of  $v_i$ .  $R^1$  eliminates the variability of  $\text{Asp}^{\text{Raw}}$ . Next, we analyze  $\text{Ext}^{\text{Raw}}$ .

**Assumption 4.3** (The assumption of  $\text{Ext}(\cdot)$ ). All  $\text{Ext}_j^{\text{Raw}}(\cdot)$  satisfy the signal strength encoding  $\text{Sig}(\cdot)$  on the physical properties of the object  $\text{Asp}_i^{\text{Raw}}$  and are scaled by a matrix  $E_j$ .

**Theorem 4.4.** When Assumption 4.3 holds, let  $\widehat{X}$  represent the normalized node feature, then  $\widehat{X}$  is TIM with respect to  $R^1$  in the dataset set  $\mathcal{D}_{GG}$ .

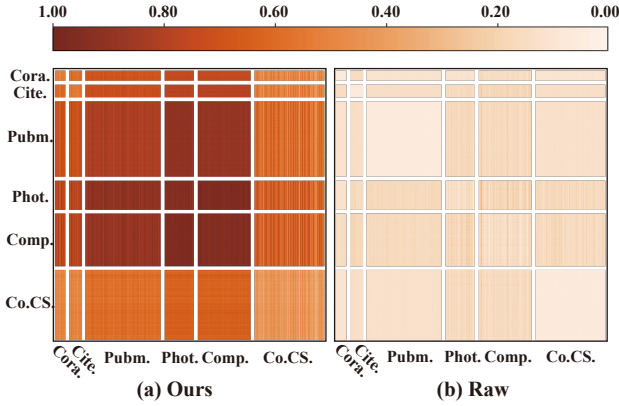
The proofs of Theorem 4.4 and Assumption 4.3 are given in Appendix A.1. Assumption 4.3 is stringent; however, the subsequent theory generally holds true even if this assumption is not fully satisfied. The  $\text{Sig}(\cdot)$  converts physical properties into a form of latent numerical signals, which are then processed into values by a matrix  $E_j$ .  $E_j$  suggests that this process is continuous. For instance, when processing temperature,  $\text{Sig}(\cdot)$  extracts the intensity of the temperature, and then  $E_j$  converts it into Celsius or Fahrenheit. Empirically, features can be categorized as discrete and continuous. Continuous features satisfy Assumption 4.3. Discrete features often arise from threshold-based sampling (e.g., age, word presence, one-hot embedding). We convert the discrete feature to the sum of a latent value and a perturbation  $\epsilon$ . If  $\mathbb{E}(\epsilon) = 0$ , it can be eliminated through sum/average operations in Corollary 4.5. If  $\mathbb{E}(\epsilon) \neq 0$ , it can be removed through noise-resistant  $\text{Ext}(\cdot)$  in Theorem 4.6. Assumption can be more flexible in practice (see Appendix A.1 for more details). Extreme cases that invalidate Assumption 4.3 (e.g., using a single feature to represent multiple classes, which is uncommon and performs poorly in most neural networks) are discussed in the Limitations section.

The  $\widehat{X}$  is TIM only with respect to  $R^1$  and is not TIM for  $R^{\text{Raw}}$ . However,  $R^1$  can help construct TIM for  $R^{\text{Raw}}$ .  $\text{Asp}^{\text{Raw}}$  and  $\mathcal{V}$  can be considered as two types of heterogeneous nodes, and  $\widehat{X}$  represents the transfer-invariant relationship between them. Therefore, we can construct a structure with the domain-invariant physical property of relationships as new  $\text{Asp}(\cdot)$  and use the domain-invariant extraction method as a new  $\text{Ext}(\cdot)$  to mine TIM for  $R^{\text{Raw}}$ .

**Corollary 4.5.** For a given dataset  $DS^{(k)} \in \mathcal{D}_{GG}$ , which contains  $X^{(k)} \in \mathbb{R}^{n^{(k)} \times d^{(k)}}$ , let  $\mathcal{A}^{(k)} = \{\text{Asp}_1^{\text{Raw}(k)}, \dots, \text{Asp}_{d^{(k)}}^{\text{Raw}(k)}\}$ . Treat elements in  $\mathcal{A}^{(k)}$  as nodes, use  $\widehat{X}^{(k)}$  to model the structure with a domain-invariant method, and apply a domain-invariant structure encoding method,  $\text{Ext}_{\mathcal{A}^{\text{Raw}}}(\cdot)$ , to extract its attributes  $X_{\text{Asp}}^{(k)}$ . Then,  $X_{\text{Asp}}$  is a TIM in  $\mathcal{D}_{GG}$  with respect to  $R^2 = \{\mathcal{A}^{(k)} | DS^{(k)} \in \mathcal{D}_{GG}\}$ .

The proof is in Appendix A.2. Here is an intuitive explanation:  $\text{Asp}_{\text{Raw}}(\cdot)$  is a graph construction method based on  $\widehat{X}$ , which is a TIM and therefore domain-invariant. Meanwhile,  $\text{Ext}_{\mathcal{A}^{\text{Raw}}}(\cdot)$  can be any domain-invariant structure encoding method. Both methods are domain-invariant and model the relationships among all  $\text{Asp}^{\text{Raw}(k)}$  within  $DS^{(k)}$ . Thus,  $X_{\text{Asp}}$  is a TIM with respect to  $R^2$ .

**Theorem 4.6.** For any dataset  $DS^{(k)} \in \mathcal{D}_{GG}$  containing  $\mathcal{A}^{(k)}$ , and for any node  $v_i \in DS^{(k)}$ , model the topology of  $(v_i, \mathcal{A}^{(k)})$  using  $\widehat{x}_{i:}^{(k)}$  as the relationship. Let  $X_{\text{Asp}}^{(k)}$  represent the attributes of  $\mathcal{A}^{(k)}$ , and



**Figure 1: Feature transfer correlations.** We calculated the similarity between nodes from general graphs. Ours are pre-trained on Cora to provided the extracted TIM features. For Figure (b), raw features are unified dimensions by SVD.

employ a domain-invariant method  $\text{Ext}_{\text{FN}}(\cdot)$  to extract its representation  $X_{\text{FN}}^{(k)}$ . Then  $X_{\text{FN}}$  is a TIM with respect to  $R^3 = \{(v_i, \mathcal{A}^{(k)}) | v_i \in \text{DS}^{(k)}, \text{DS}^{(k)} \in \mathcal{D}_{\text{GG}}\}$  in the dataset set  $\mathcal{D}_{\text{GG}}$ .

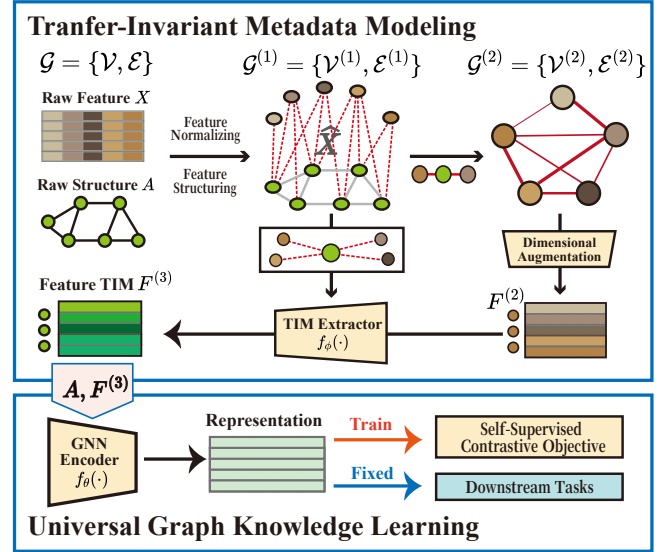
The proof of Theorem 4.6 is in Appendix A.3. Combining all these results, we have theoretically mined TIM from node features. Furthermore, in Theorem 4.6, the original node features  $X$  are largely retained within the structure between  $\mathcal{V}$  and  $\mathcal{A}^{(k)}$ , resulting in low information loss. Theorem 4.6 also unifies dimensions through augmentation for each feature (not dimensional reduction). Hence, Theorem 4.6 adequately addresses the two main challenges of node feature universality raised in Section 1.

**Experimental Validation.** To validate our theory, we evaluated the transfer correlations of raw data and the extracted transfer-invariant metadata (TIM) based on Theorem 4.6, as illustrated in Figure 1. Specifically, we calculate the similarity between each node across different graph datasets. Across all datasets, ours TIM exhibited significantly greater similarity (approximately 10x the raw data). This robustly demonstrates that TIM extracted based on Theorem 4.6 indeed possesses domain-invariant semantics. Additionally, these findings offer theoretical support for the superior performance of our proposed approach in downstream tasks.

## 5 Method

Based on the theoretical foundations in Section 4, we proposed a Transfer-Invariant Graph foundation model (TIG). The overview of the model is shown in Figure 2.

**Feature TIM Modeling.** As described in Section 4, given an input graph  $\mathcal{G} \in \mathcal{D}_{\text{GG}}$ , we first apply Min-Max normalization to obtain the TIM representing the node- $\text{Asp}^{\text{Raw}}$  relationship,  $\hat{X}$ . Based on this, we structure the features into a new graph  $\mathcal{G}^{(1)} = (\mathcal{V}^{(1)}, \mathcal{E}^{(1)})$ , where  $\mathcal{V}^{(1)} = \mathcal{V} \cup \text{Asp}^{\text{Raw}}$ . All nodes in  $\mathcal{G}^{(1)}$  have no features. To prevent overly large or small edge weights, we set  $e_{ij}^{(1)} = \hat{x}_{ij} / \sum_j \hat{x}_{ij}$ , which, according to Corollary 4.2, does not alter the properties of the TIM. Next, we model all nodes in the set



**Figure 2: The overview of our TIG.**

$\text{Asp}^{\text{Raw}}$  together. As in Theorem 4.5, we model the relationships between any two nodes in  $\text{Asp}^{\text{Raw}}$  through Node-Asp-Node paths, generating the graph  $\mathcal{G}^{(2)} = (\mathcal{V}^{(2)}, \mathcal{E}^{(2)})$  with  $\mathcal{V}^{(2)} = \text{Asp}^{\text{Raw}}$ , and edge weights as follows:

$$e_{ij}^{(2)} = \text{Min-Max} \left( \sum_{k \in \mathcal{V}} e_{ik}^{(1)} e_{kj}^{(1)} \right), \quad (6)$$

here, we apply Min-Max normalization to maintain the similarity of structural weights. For  $\mathcal{G}^{(2)}$ , inspired by existing methods [27], we use an SVD-based algorithm to embed the structure as dimensional augmentation:

$$U, \Lambda, \Gamma = \text{SVD}(\mathcal{E}^{(2)}), F^{(2)} = U\sqrt{\Lambda} + \Gamma\sqrt{\Lambda}, \quad (7)$$

where  $U, \Gamma \in \mathbb{R}^{n \times s}$ ,  $\Lambda \in \mathbb{R}^{s \times s}$  are from the SVD of the adjacency matrix of  $\mathcal{G}^{(2)}$ . Since  $s$  are defined by a hyperparameter, the dimension of  $F^{(2)}$  can be consistent across different datasets. Based on Theorem 4.5,  $F^{(2)}$  is TIM. We then combine  $F^{(2)}$  with  $\mathcal{G}^{(1)}$  to generate the final TIM embedding, as shown in the equation below:

$$F^{(3)} = f_{\phi}(F^{(2)}, \mathcal{E}^{(1)}) = A^{(1)} F^{(2)} W_{\phi}, \quad (8)$$

where  $A^{(1)}$  is the adjacency matrix of  $\mathcal{E}^{(1)}$ , and  $W_{\phi}$  is a learnable matrix to eliminate disturbances when Assumption 4.3 may not hold, as detailed in Appendix A.1. Consequently, for each node, we extract a TIM embedding representing the relationship between the node and all objects in  $\text{Asp}^{\text{Raw}}$ . It is worth noting that the dimension of  $F^{(2)}$  is consistent across different datasets, and all the original feature information is embedded in  $A^{(1)}$ . As a result, the  $F^{(3)}$  fully unifies the original node features dimensions.

**Self-Supervised Universal Graph Learning.** We adopt the existing graph pre-training paradigm to achieve cross-dataset self-supervised learning of graph patterns. TIG builds on the representation scattering concept proposed in SGRL [5], a recent graph contrastive learning method. To reduce model complexity, we omit its asymmetric topological aggregation framework. Specifically, given the original graph structure  $A$  and the extracted TIM features

**Table 1: Performance on cross-domain node classification. Methods with “\*” are reported from the original paper. FUG-C (ACG) means the average performance of FUG [31] which were pre-trained and tested on the datasets as we did. TIG (Cora) means the TIG pre-trained on Cora dataset.**

| Target datasets | Cora         | CiteSeer     | PubMed       | Photo        | Computers    | CS           | Physics      |
|-----------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| OFA*            | 75.90 ± 1.26 | -            | 78.25 ± 0.71 | -            | -            | -            | -            |
| GraphControl*   | -            | -            | -            | 89.66 ± 0.56 | -            | -            | 94.31 ± 0.12 |
| FUG-C (AVG)     | 82.45 ± 0.87 | 67.50 ± 1.98 | 84.48 ± 0.16 | 91.58 ± 2.87 | 84.66 ± 4.85 | 92.52 ± 0.19 | 95.36 ± 0.11 |
| TIG (Cora)      | 84.85 ± 2.29 | 70.96 ± 2.17 | 86.05 ± 0.41 | 92.44 ± 1.17 | 88.65 ± 0.60 | 93.11 ± 0.55 | 95.84 ± 0.31 |
| TIG (CiteSeer)  | 82.57 ± 3.19 | 70.96 ± 3.21 | 86.16 ± 0.62 | 92.47 ± 0.75 | 88.63 ± 0.68 | 92.94 ± 0.60 | 95.73 ± 0.29 |
| TIG (PubMed)    | 83.46 ± 2.99 | 71.02 ± 1.90 | 85.96 ± 0.53 | 92.65 ± 0.97 | 88.58 ± 0.68 | 93.17 ± 0.44 | 95.70 ± 0.36 |
| TIG (Photo)     | 83.09 ± 2.76 | 71.50 ± 2.34 | 86.17 ± 0.29 | 92.76 ± 0.85 | 88.46 ± 0.67 | 93.03 ± 0.65 | 95.83 ± 0.29 |
| TIG (Computers) | 83.53 ± 3.83 | 72.34 ± 2.14 | 86.35 ± 0.38 | 92.47 ± 0.70 | 88.62 ± 0.74 | 93.04 ± 0.53 | 95.79 ± 0.26 |
| TIG (CS)        | 82.50 ± 3.49 | 71.32 ± 1.79 | 86.17 ± 0.57 | 92.55 ± 1.08 | 88.65 ± 0.93 | 93.21 ± 0.52 | 95.71 ± 0.33 |
| TIG (Physics)   | 83.68 ± 2.32 | 71.62 ± 2.67 | 86.32 ± 0.20 | 92.60 ± 0.75 | 88.50 ± 0.78 | 93.21 ± 0.52 | 95.74 ± 0.34 |

$F^{(3)}$ , we first generate its representation using a graph neural network  $f_\theta(\cdot)$ ,  $Z = f_\theta(A, F^{(3)})$ . Next, we train the  $f_\theta(\cdot)$  and  $f_\phi(\cdot)$  using the existing graph contrastive objective:

$$\mathcal{L}(v_i) = -\|\text{Norm}(z_i) - \text{Mean}(\text{Norm}(Z))\|_2 + \frac{1}{|\mathcal{N}(v_i)|} \sum_{v_j \in \mathcal{N}(v_i)} \|z_i - z_j\|_2, \quad (9)$$

where  $\text{Mean}(\cdot)$  computes the mean vector of all node representations, and  $\text{Norm}(\cdot)$  denotes vector normalization. This objective directly aligns each node representation with its neighbors while encouraging dispersion across the graph.

## 6 Experiment

To thoroughly validate the proposed TIM theory and TIG model, we conducted experiments on seven widely used node-level datasets (including Cora, CiteSeer, PubMed, Photo, Computer, CS, Physics), and six molecular graph-level datasets (including BBBP, Tox21, ToxCast, ClinTox, MUV, HIV). Considering that TIG is a graph foundational model, we completely evaluate TIG in various pre-training and fine-tuning scenarios, which include: (1) Cross-domain pretraining and finetuning scenarios, (2) Cross-domain one-shot scenarios (pseudo zero-shot scenario), (3) Cross-domain few-shot scenarios, and (4) Transfer scenario from citation graph to the molecular graphs. Considering that model performance in cross-domain settings may be influenced by various environmental factors, which can potentially prevent existing models from achieving optimal performance. Therefore, for the compared methods, we strive to report the most advanced and reliable results across all scenarios from [3, 29, 31, 37]. More experimental setting details can be found in Appendix B. In all tables, the best results are bolded, and the second-best results are underlined. *In addition, we have supplemented experiments on graph-level tasks in Appendix C.* Our code is available at <https://github.com/hedongxiao-tju/TIG>.

### 6.1 Performance Analysis

**Cross-Domain Scenarios.** As shown in Table 1, in cross-domain settings, our TIG consistently outperforms all existing advanced

methods when pre-trained on any single dataset. This validates both the correctness of our TIM theory and the strong transferability and generalization ability of the TIG model. Table 1 exhibits an interesting phenomenon: in more than half of the cases, the highest performance is not achieved when the pre-training and test datasets are the same. This indicates that our model can effectively generalize knowledge across different datasets. Therefore, the model can effectively utilize the knowledge learned from other datasets to enhance its performance. The TIM guarantees this transfer ability. Furthermore, the performance differences across different training datasets are minimal, indicating that these datasets share similar feature semantics. This further confirms the consistency of the extracted TIM across domains. Additionally, TIG outperforms FUG, demonstrating that although FUG can unify node feature dimensions, the encoded semantics cannot be invariant across different domains. Additionally, TIG outperforms GraphControl, which constructs a new structure using feature similarity, resulting in a significant loss of information. These results further confirm our view in Discussion 2 of Section 4.

**Cross-Domain 1-shot (pseudo zero-shot) Scenarios.** The results are shown in Table 2. In this scenario, TIG does not employ any prompt-tuning strategy. Instead, it just randomly selects a node from each class as the class prototype vector and predicts the node class based on its similarity to the prototype. Since no fine-tuning is involved, we refer to this setting as a pseudo zero-shot scenario. As shown in Table 2, despite other methods employing carefully designed prompt-tuning strategies and leveraging more training data than TIG (TIG is pretrained using only a single dataset, whereas other methods utilize all four remaining datasets), TIG achieves significant improvements on four out of five datasets and performs comparably to MDGPT on the remaining one. This demonstrates TIG’s strong generalization ability in few-shot settings. TIG significantly outperforms all methods except MDGPT. Most of these competing methods rely on arbitrary data pre-processing to unify node feature dimensions. However, in cross-domain few-shot scenarios, this approach struggles when faced with new node features that carry different semantics, leading to accuracy degradation. In contrast, MDGPT employs a well-designed prompt-tuning strategy

**Table 2: Performance on cross-domain one-shot scenarios (pseudo zero-shot scenarios). Methods with “\*” are reported from [29]. “-AVG”, “-MIN”, “-MAX” denote the average, min and max performance of our model pre-trained on one of four cross-domain datasets. PT means prompt tuning.**

| Target datasets | PT | Cora                 | CiteSeer            | PubMed              | Photo                | Computers            |
|-----------------|----|----------------------|---------------------|---------------------|----------------------|----------------------|
| GPPT*           | ✓  | 15.37 ± 4.51         | 23.24 ± 2.94        | 36.56 ± 5.31        | 16.19 ± 4.73         | 19.22 ± 8.71         |
| GraphPrompt*    | ✓  | 35.90 ± 7.10         | 32.76 ± 7.66        | 43.34 ± 10.66       | 49.88 ± 8.31         | 43.03 ± 10.35        |
| GPF*            | ✓  | 37.84 ± 11.07        | 37.61 ± 8.87        | 46.36 ± 7.48        | 49.42 ± 7.04         | 37.00 ± 6.52         |
| GCOPE*          | ✓  | 33.38 ± 6.86         | 35.56 ± 6.81        | 42.10 ± 8.07        | 48.52 ± 7.78         | 40.22 ± 7.82         |
| MDGPT*          | ✓  | <u>42.26 ± 10.18</u> | <u>42.40 ± 9.26</u> | <u>49.82 ± 8.38</u> | <u>64.82 ± 10.53</u> | <b>49.77 ± 11.00</b> |
| TIG-AVG         | ×  | <b>48.43 ± 1.13</b>  | <b>45.56 ± 0.58</b> | <b>52.80 ± 0.50</b> | <b>65.68 ± 1.01</b>  | <u>49.35 ± 0.96</u>  |
| TIG-MIN         | ×  | 46.85 ± 8.55         | 45.05 ± 9.46        | 52.16 ± 9.91        | 64.49 ± 9.50         | 48.34 ± 9.95         |
| TIG-MAX         | ×  | 49.95 ± 9.22         | 46.48 ± 9.14        | 53.38 ± 11.00       | 67.29 ± 9.91         | 50.87 ± 11.09        |

**Table 3: Performances on cross-domain few-shot scenarios. Methods with “\*” are reported from [37]. Methods with “-MAX” means report their best performance with different pre-train models.**

|              |                  | 3-shot              |                     |                     | 5-shot              |                     |                     |
|--------------|------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|
|              |                  | Cora                | CiteSeer            | PubMed              | Cora                | CiteSeer            | PubMed              |
| In-domain    | Fine-tuning-MAX* | 51.97 ± 2.84        | 45.08 ± 2.09        | 65.40 ± 3.00        | 62.66 ± 3.55        | 39.54 ± 3.54        | <b>70.91 ± 4.87</b> |
|              | GPPT-MAX*        | 43.84 ± 6.11        | 42.34 ± 8.31        | <b>67.43 ± 2.96</b> | 51.98 ± 3.43        | 45.77 ± 7.41        | <u>66.97 ± 3.70</u> |
|              | AIO-MAX*         | 48.09 ± 4.83        | 48.09 ± 8.18        | 65.79 ± 5.79        | 30.45 ± 0.19        | 27.93 ± 10.59       | 46.16 ± 15.83       |
| Cross-domain | TIG (Cora)       | -                   | <b>52.04 ± 5.75</b> | <u>65.92 ± 6.55</u> | -                   | <u>59.71 ± 3.97</u> | 64.60 ± 5.55        |
|              | TIG (CiteSeer)   | <u>52.99 ± 5.48</u> | -                   | 59.47 ± 6.64        | <b>71.21 ± 3.05</b> | -                   | 65.74 ± 5.63        |
|              | TIG (PubMed)     | <b>56.19 ± 5.89</b> | <u>51.67 ± 5.71</u> | -                   | <u>69.13 ± 3.44</u> | <b>60.02 ± 3.99</b> | -                   |

to align domains, allowing it to adapt to differences between up-stream and downstream datasets. TIG, however, inherently encodes cross-domain invariant node features, enabling it to generalize effectively even without additional prompt-tuning.

**Cross-Domain Few-Shot Scenarios.** The results in Table 3 compare cross-domain trained TIG directly with in-domain trained models in few-shot settings, as competing methods exhibit extremely low performance in cross-domain scenarios. To show the best performance of the compared methods, we report the highest accuracy achieved by the recent outstanding benchmark [37], where each prompt-tuning framework is paired with its most effective pre-trained model. Even under this unfavorable comparison, TIG still surpasses existing methods on two out of three datasets and achieves comparable performance on the PubMed dataset, further validating its strong generalization ability. Additionally, a recurring observation from Tables 1, 2, and 3 is that different training datasets have a minimal impact on model performance. This is because TIM extraction unifies node features into a shared space with consistent semantics, ensuring transfer-invariant node features. As a result, TIG effectively leverages knowledge learned during pre-training, maintaining its strong generalization ability even in cross-domain scenarios. Furthermore, in contrast to Table 1, where many cross-domain results of TIG were superior to in-domain results of TIG, Table 3 shows that TIG generally performs better when trained and tested within the same domain. We hypothesize that when

supervision is extremely limited, some randomly selected labeled nodes may be of low quality, meaning their representations lie near class boundaries. In-domain trained models adapt better to these noisy supervision signals, resulting in improved performance.

**From Citation Graph to Molecular Graph Transfer Scenarios.** To comprehensively assess TIG’s cross-domain generalization, we evaluated its performance when transferring from citation datasets to molecular datasets. For our experimental setup, we followed excellent work GPF [3], which focuses on domain-specific pretraining and fine-tuning, and compared our cross-domain results with those reported for within-domain graph classification which favors their methods. Besides, to ensure a fair comparison, the compared GPF uses GCL as the backbone—identical to our proxy task—and reports both its direct fine-tuning results (which are consistent with our setup) and its results with GPF-based promoting. It is worth noting that the above experimental settings heavily favor the baseline methods compared. To be specific, they perform pre-training on the molecular dataset ZINC15, which is within-domain, whereas our model is pretrained on the citation dataset Cora, which is cross-domain. Additionally, their model framework is more tailored for graph classification, whereas our approach consistently employs a two-layer GCN, identical to the one used for node classification. The results are shown in Table 4. We report both TIG-Rebuild, where hyperparameters are individually tuned for each transfer scenario (consistent with the GPF setting), and TIG, which

**Table 4: Fifty-shot test ROC-AUC (%) performance on molecular prediction benchmarks. The experimental settings follow those used in GPF [3]. The results for GCL-FT, GCL-GPF, and GCL-GPF-plus are reported from [3]. FT refers to directly fine-tuning a downstream classifier, while GPF and GPF-plus refer to using the promotion algorithms proposed in by GPF. “-Rebuild” implies that the hyper-parameters for different datasets are not consistent, following GPF.**

| Method                | BBBP                | Tox21               | ToxCast             | ClinTox             | MUV                 | HIV                 |
|-----------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|
| GCL-FT (ZINC15)       | 54.40 ± 2.87        | 48.35 ± 1.67        | 50.29 ± 0.19        | 54.05 ± 4.16        | 46.73 ± 1.88        | 60.05 ± 3.80        |
| GCL-GPF (ZINC15)      | 53.87 ± 2.17        | <u>50.58 ± 0.49</u> | 52.64 ± 0.50        | <u>64.44 ± 4.64</u> | 47.22 ± 3.55        | <b>64.86 ± 1.29</b> |
| GCL-GPF-plus (ZINC15) | <u>55.89 ± 1.58</u> | 50.14 ± 1.09        | <u>53.25 ± 0.95</u> | <b>65.22 ± 4.51</b> | <u>47.88 ± 1.77</u> | <u>63.99 ± 1.60</u> |
| TIG-Rebuild (Cora)    | <b>63.70 ± 0.18</b> | <b>54.33 ± 0.06</b> | <b>53.29 ± 0.16</b> | 54.39 ± 4.49        | <b>50.60 ± 7.48</b> | 61.16 ± 0.67        |
| TIG (Cora)            | 59.70 ± 1.53        | 53.18 ± 0.30        | 53.12 ± 0.14        | 54.44 ± 0.35        | 46.16 ± 0.40        | 60.00 ± 0.70        |

**Table 5: Ablation study of our feature TIM mining method.**

|                                  |                           | Cora         | CiteSeer     | PubMed       | Photo        | Computers    | CS           | Physics      |
|----------------------------------|---------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| Continuous + SVD + GCN (Ours)    |                           | 85.51 ± 2.09 | 73.47 ± 0.78 | 86.88 ± 0.61 | 93.99 ± 0.87 | 89.01 ± 0.70 | 93.49 ± 0.63 | 95.86 ± 0.47 |
| $\mathcal{G}^{(2)}$ Construction | Discrete                  | 84.26 ± 2.30 | 73.11 ± 1.43 | 86.79 ± 0.35 | 93.41 ± 1.04 | 88.97 ± 0.73 | 93.30 ± 0.47 | 95.80 ± 0.41 |
|                                  | Discrete with threshold   | 85.00 ± 2.64 | 72.34 ± 1.40 | 85.76 ± 0.59 | 93.59 ± 0.98 | 88.74 ± 0.77 | 93.14 ± 0.66 | 95.85 ± 0.38 |
|                                  | Continuous with threshold | 85.29 ± 2.06 | 71.98 ± 1.54 | 85.95 ± 0.66 | 93.31 ± 1.17 | 88.53 ± 0.64 | 93.24 ± 0.53 | 95.82 ± 0.30 |
| Augmentation                     | PCA                       | 84.78 ± 2.93 | 68.92 ± 1.69 | 86.28 ± 0.77 | 91.29 ± 0.62 | 89.32 ± 0.59 | 92.76 ± 0.29 | 95.78 ± 0.37 |
|                                  | Node2Vec                  | 83.82 ± 2.06 | 70.96 ± 2.50 | 83.78 ± 0.60 | 92.05 ± 1.23 | 87.70 ± 0.24 | 91.12 ± 0.92 | 95.47 ± 0.44 |
| TIM Extractor                    | Two layers of GCN         | 84.71 ± 3.10 | 71.92 ± 1.29 | 86.67 ± 0.57 | 93.59 ± 1.27 | 88.75 ± 0.73 | 93.22 ± 0.55 | 95.70 ± 0.45 |
|                                  | GCN + Linear              | 85.15 ± 2.88 | 71.98 ± 1.15 | 86.54 ± 0.80 | 93.49 ± 1.28 | 88.88 ± 0.82 | 93.17 ± 0.60 | 95.81 ± 0.42 |

uses a unified hyperparameter configuration across all scenarios. We observed: First, cross-domain TIG achieves performance comparable to in-domain GPF, demonstrating its ability to effectively unify graph representations across heterogeneous domains. Second, TIG-Rebuild consistently outperforms GCL-FT, indicating that without specialized fine-tuning strategies, cross-domain pretrained TIG have the potential to surpass models trained within the target domain. Third, the comparable performance between TIG-Rebuild and TIG highlights the strong generalization ability of our TIG, showing that it is not sensitive to scenario-specific hyperparameter tuning.

## 6.2 Model Analysis

To evaluate the impact of different Asp and Ext configurations on TIM extraction, we conducted an ablation study on different TIM mining methods, as shown in Table 5. Specifically, we tested the graph ( $\mathcal{G}^{(2)}$ ) construction method, the dimensional augmentation method, and the implementation of TIM extractor. From Table 5, we derive two key observations. First, the choice of details has a relatively minor impact on model performance. This aligns with the theoretical findings in Section 5, which state that if the methods achieve cross-domain invariance, they can successfully extract TIM. This further substantiates that TIG’s success primarily stems from the proposed TIM theory, rather than specific implementation details. Second, among all factors, dimensional augmentation has the most significant impact on model performance. According to Theorem 4.5, this technique converts topological signals into numerical embeddings. However, as mentioned in Discussion 1 of Section 4.1, this extraction process inevitably incurs information

loss. Thus, selecting an appropriate augmentation may enhance the extraction of useful information, resulting in improved model performance. Our SVD-based approach effectively captures structural information, contributing to superior results.

## 7 Conclusion and Future Work

In this paper, we focus on exploring the Transfer-Invariant Metadata (TIM) within general graph node features, providing insights for graph foundation models. We propose a TIM theory to universally extract TIM that shares semantic spaces across different datasets and introduce the Transfer-Invariant Graph foundation model (TIG). TIG demonstrates outstanding performance across various cross-domain pre-training and fine-tuning scenarios, even outperforming other in-domain models in few-shot settings. We hope that the TIM theory proposed in this work will further support the development and design of graph foundation models in the future. Additionally, upon acceptance of this paper, we will release a pre-trained TIG model that can be directly applied to extract universal representations for various general graphs, enabling the development of diverse graph-based downstream tasks.

## Acknowledgments

This work is supported by the National Natural Science Foundation of China (No. 62422210, No. 62276187, No. 62302333, No.62372323 and No. 92370111), the National Key Research and Development Program of China (No. 2023YFC3304503), the Hong Kong RGC theme-based Strategic Target Grant Scheme (STG STG1/M-501/23-N), the Hong Kong Global STEM Professor Scheme, and the Hong Kong Jockey Club Charities Trust.

## References

- [1] Piotr Bielak, Tomasz Kajdanowicz, and Nitesh V. Chawla. 2022. Graph Barlow Twins: A self-supervised representation learning framework for graphs. *Knowl. Based Syst.* 256 (2022), 109631. <https://doi.org/10.1016/j.knosys.2022.109631>
- [2] Kaize Ding, Jianling Wang, Jundong Li, Kai Shu, Chenghao Liu, and Huan Liu. 2020. Graph prototypical networks for few-shot learning on attributed networks. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 295–304.
- [3] Taoran Fang, Yunchao Zhang, Yang Yang, Chunping Wang, and Lei Chen. 2023. Universal Prompt Tuning for Graph Neural Networks. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.).
- [4] Mikhail Galkin, Xinyu Yuan, Hesham Mostafa, Jian Tang, and Zhaocheng Zhu. 2024. Towards Foundation Models for Knowledge Graph Reasoning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net. <https://openreview.net/forum?id=jVEoydFOL9>
- [5] Dongxiao He, Lianze Shan, Jitao Zhao, Hengrui Zhang, Zhen Wang, and Weixiong Zhang. 2024. Exploitation of a Latent Mechanism in Graph Contrastive Learning: Representation Scattering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [6] Thomas N. Kipf and Max Welling. 2016. Variational Graph Auto-Encoders. *CoRR* abs/1611.07308 (2016). [arXiv:1611.07308](http://arxiv.org/abs/1611.07308)
- [7] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=SJU4ayYgl>
- [8] Nils Kriege and Petra Mutzel. 2012. Subgraph matching kernels for attributed graphs. *arXiv preprint arXiv:1206.6483* (2012).
- [9] Hao Liu, Jiarui Feng, Lecheng Kong, Ningyue Liang, Dacheng Tao, Yixin Chen, and Muhan Zhang. 2023. One for All: Towards Training One Graph Model for All Classification Tasks. *CoRR* abs/2310.00149 (2023). [arXiv:2310.00149](https://arxiv.org/abs/2310.00149)
- [10] Jiawei Liu, Cheng Yang, Zhiyuan Lu, Junze Chen, Yibo Li, Mengmei Zhang, Ting Bai, Yuan Fang, Lichao Sun, Philip S Yu, et al. 2023. Towards graph foundation models: A survey and beyond. *arXiv preprint arXiv:2310.11829* (2023).
- [11] Zemin Liu, Xingtong Yu, Yuan Fang, and Xinming Zhang. 2023. GraphPrompt: Unifying Pre-Training and Downstream Tasks for Graph Neural Networks. In *Proceedings of the ACM Web Conference 2023, WWW 2023, Austin, TX, USA, 30 April 2023 - 4 May 2023*, Ying Ding, Jie Tang, Juan F. Sequeda, Lora Aroyo, Carlos Castillo, and Geert-Jan Houben (Eds.). ACM, 417–428. doi:10.1145/3543507.3583386
- [12] Haitao Mao, Zhikai Chen, Wenzhuo Tang, Jianan Zhao, Yao Ma, Tong Zhao, Neil Shah, Mikhail Galkin, and Jiliang Tang. [n. d.]. Position: Graph Foundation Models Are Already Here. In *Forty-first International Conference on Machine Learning*.
- [13] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781* (2013).
- [14] Jiezhong Qiu, Qibin Chen, Yuxiao Dong, Jing Zhang, Hongxia Yang, Ming Ding, Kuansan Wang, and Jie Tang. 2020. GCC: Graph Contrastive Coding for Graph Neural Network Pre-Training. In *KDD '20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, CA, USA, August 23–27, 2020*, Rajesh Gupta, Yan Liu, Jiliang Tang, and B. Aditya Prakash (Eds.). ACM, 1150–1160. doi:10.1145/3394486.3403168
- [15] Ryan Rossi and Nesreen Ahmed. 2015. The network data repository with interactive graph analytics and visualization. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 29.
- [16] Yangyi Shen, Jincheng Zhou, Beatrice Bevilacqua, Joshua Robinson, Charilaos I. Kanatsoulis, Jure Leskovec, and Bruno Ribeiro. 2025. Zero-Shot Generalization of GNNs over Distinct Attribute Domains. In *Forty-second International Conference on Machine Learning, ICLR 2025, Vancouver, BC, Canada, July 13–19, 2025*. OpenReview.net. <https://openreview.net/forum?id=R9iBZen9aY>
- [17] Li Sun, Zhenhao Huang, Suyang Zhou, Qiqi Wan, Hao Peng, and Philip Yu. 2025. Riemanngm: Learning a graph foundation model from riemannian geometry. In *Proceedings of the ACM on Web Conference 2025*. 1154–1165.
- [18] Mingchen Sun, Kaixiong Zhou, Xin He, Ying Wang, and Xin Wang. 2022. GPPT: Graph Pre-training and Prompt Tuning to Generalize Graph Neural Networks. In *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*, Aidong Zhang and Huzefa Rangwala (Eds.). ACM, 1717–1727. doi:10.1145/3534678.3539249
- [19] Xiangguo Sun, Hong Cheng, Jia Li, Bo Liu, and Jihong Guan. 2023. All in One: Multi-Task Prompting for Graph Neural Networks. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6–10, 2023*, Ambuj K. Singh, Yizhou Sun, Leman Akoglu, Dimitrios Gunopulos, Xifeng Yan, Ravi Kumar, Fatma Ozcan, and Jieping Ye (Eds.). ACM, 2120–2131. doi:10.1145/3580305.3599256
- [20] Xiangguo Sun, Hong Cheng, Jia Li, Bo Liu, and Jihong Guan. 2024. All in One: Multi-task Prompting for Graph Neural Networks (Extended Abstract). In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3–9, 2024*. ijcai.org, 8460–8465. <https://www.ijcai.org/proceedings/2024/942>
- [21] Shantanu Thakoor, Corentin Tallec, Mohammad Gheshlaghi Azar, Rémi Munos, Petar Velicković, and Michal Valko. 2021. Bootstrapped representation learning on graphs. In *ICLR 2021 Workshop on Geometrical and Topological Representation Learning*.
- [22] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=rJXmpikCZ>
- [23] Petar Velickovic, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R. Devon Hjelm. 2019. Deep Graph Infomax. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019*. OpenReview.net. <https://openreview.net/forum?id=rklz9iAcKQ>
- [24] Sebastian Vlais, Theresia Conrad, Christian Tokarski-Schnelle, Mika Gustafsson, Uta Dahmen, Reinhard Guthke, and Stefan Schuster. 2018. ModuleDiscoverer: Identification of regulatory modules in protein-protein interaction networks. *Scientific reports* 8, 1 (2018), 433. <https://doi.org/10.1038/s41598-017-18370-2>
- [25] Kai Wang and Siquang Luo. 2024. Towards Graph Foundation Models: The Perspective of Zero-shot Reasoning on Knowledge Graphs. *CoRR* abs/2410.12609 (2024). [arXiv:2410.12609](https://arxiv.org/abs/2410.12609)
- [26] Shu Wu, Yuyuan Tang, Yanqiao Zhu, Liang Wang, Xing Xie, and Tieniu Tan. 2019. Session-based recommendation with graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 346–353.
- [27] Lianghao Xia and Chao Huang. 2024. Anygraph: Graph foundation model in the wild. (2024).
- [28] Pinar Yanardag and SVN Vishwanathan. 2015. Deep graph kernels. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*. 1365–1374.
- [29] Xingtong Yu, Chang Zhou, Yuan Fang, and Xinming Zhang. 2024. Text-Free Multi-domain Graph Pre-training: Toward Graph Foundation Models. *CoRR* abs/2405.13934 (2024). [arXiv:2405.13934](https://arxiv.org/abs/2405.13934)
- [30] Haihong Zhao, Aochuan Chen, Xiangguo Sun, Hong Cheng, and Jia Li. 2024. All in One and One for All: A Simple yet Effective Method towards Cross-domain Graph Pretraining. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25–29, 2024*. ACM, 4443–4454. doi:10.1145/3637528.3671913
- [31] Jitao Zhao, Di Jin, Meng Ge, Lianze Shan, Xin Wang, Dongxiao He, and Zhiyong Feng. 2024. FUG: Feature-Universal Graph Contrastive Pre-training for Graphs with Diverse Node Features. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [32] Jianan Zhao, Hesham Mostafa, Mikhail Galkin, Michael Bronstein, Zhaocheng Zhu, and Jian Tang. 2024. Graphany: A foundation model for node classification on any graph. *arXiv preprint arXiv:2405.20445* (2024).
- [33] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).
- [34] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI open* 1 (2020), 57–81.
- [35] Yun Zhu, Yaoke Wang, Haizhou Shi, Zhenshuo Zhang, Dian Jiao, and Siliang Tang. 2024. GraphControl: Adding Conditional Control to Universal Graph Pre-trained Models for Graph Domain Transfer Learning. In *Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, May 13–17, 2024*. ACM, 539–550. doi:10.1145/3589334.3645439
- [36] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2020. Deep Graph Contrastive Representation Learning. *CoRR* abs/2006.04131 (2020). [arXiv:2006.04131](https://arxiv.org/abs/2006.04131)
- [37] Chenyi Zi, Haihong Zhao, Xiangguo Sun, Yiqing Lin, Hong Cheng, and Jia Li. 2024. ProG: A Graph Prompt Learning Benchmark. *CoRR* abs/2406.05346 (2024). [arXiv:2406.05346](https://arxiv.org/abs/2406.05346)

## A Supplement of TIM Theory

### A.1 The Proof of Theorem 4.4 and The Discussion on Assumption 4.3

**Proof of Theorem 4.4.** As mentioned in Assumption 4.3,  $x_{ij} = \text{Sig}(v_i, \text{Asp}_j^{\text{Raw}})E_j$ , where  $\text{Sig}(\cdot)$  is consistent across datasets. For simplicity, we consider  $t_{ij} = \text{Sig}(v_i, \text{Asp}_j^{\text{Raw}})$  and the normalization

method is Min-Max normalization. Then, we have:

$$\begin{aligned}
\hat{x}_{ij} &= \text{Min-Max}(x_{ij}) = \text{Min-Max}[t_{ij}E_j] \\
&= \frac{t_{ij}E_j - \text{Min}(t_{ij}E_j)}{\text{Max}(t_{ij}E_j) - \text{Min}(t_{ij}E_j)} = \frac{[t_{ij} - \text{Min}(t_{ij})]E_j}{[\text{Max}(t_{ij}) - \text{Min}(t_{ij})]E_j} \\
&= ([t_{ij} - \text{Min}(t_{ij})]E_j)([\text{Max}(t_{ij}) - \text{Min}(t_{ij})]E_j)^{-1} \\
&= [t_{ij} - \text{Min}(t_{ij})][\text{Max}(t_{ij}) - \text{Min}(t_{ij})]^{-1} \\
&= \frac{[t_{ij} - \text{Min}(t_{ij})]K}{[\text{Max}(t_{ij}) - \text{Min}(t_{ij})]K} \\
&= \text{Min-Max}(t_{ij}) = \text{Min-Max}[\text{Sig}(v_i, \text{Asp}_j^{\text{Raw}})K]
\end{aligned} \tag{10}$$

here, we use two mathematical tricks to proof. First, we calculate the pseudo-inverse matrix of  $E_j$ . Second, we assume a universal transformation  $K$  that converts physical signal strengths into numerical values. In above equation,  $\text{Min-Max}(\cdot)$  is consistent across datasets,  $\text{Sig}(\cdot)K$  is consistent across datasets. When we consider the  $\text{Sig}(\cdot)K$  as a kind of  $\text{Asp}(\cdot)$ , and  $\text{Min-Max}(\cdot)$  as a kind of  $\text{Ext}(\cdot)$ ,  $\hat{X}$  is TIM with respect to  $R^1 = \{(v_i, \text{Asp}_j^{\text{Raw}}) | v_i \in R^{\text{Raw}}, \text{Asp}_j^{\text{Raw}} \in \text{Asp}^{\text{Raw}}\}$  in the dataset set  $\mathcal{D}_{\text{GG}}$ . For other normalization methods,  $E_j$  can also be eliminated, thereby transforming the cross-dataset variant in  $E_j$  into an invariant  $\text{Ext}(\cdot)$ .

**Discussion on Assumption 4.3.** Assumption 4.3 imposes constraints on how numerical datasets are mathematically modeled and how an object  $R_i$  is expressed in terms of its  $\text{Asp}(\cdot)$  features. Specifically, it assumes that for each physical property  $\text{Asp}_j(\cdot)$ , there exists a measurable representation, such as length, width, or temperature. The corresponding physical signal strength encoding is denoted as  $\text{Sig}(\cdot)$  which is consistent among common physical properties, while  $E_j$  encodes this signal's intensity into a mathematically defined range. For example, if  $\text{Asp}_j(\cdot)$  represents temperature, then  $\text{Sig}_j(\cdot)$  extracts the temperature of object  $R_i$ , while  $E_j$  scales this measurement into either the Celsius or Fahrenheit space, depending on the chosen metric system. Assumption 4.3 defines  $E_j$  as a matrix, implying that it satisfies linear and continuous transformations, which hold for most continuously measured data. However, when the data is discontinuous, an additional perturbation term  $\epsilon$  is introduced under the data after transformation  $E_j$ , similar to the idea in Gumbel-Max Trick. In Theorem 4.4, data discontinuity invalidates the result because  $\epsilon_j$  varies across different  $\text{Asp}_j(\cdot)$  and cannot be eliminated. In contrast, Corollary 4.5 remains valid because it models the internal relationships of  $\text{Asp}$  within the entire feature  $X$ , allowing the summation process to eliminate the Gaussian-distributed perturbation  $\epsilon$  i.e. the expectation of  $\epsilon$  is 0. Furthermore, in Theorem 4.6, introducing a denoising function  $\text{Ext}_{\text{FN}}(\cdot)$  or using discrete-to-continuous fitting methods ensures that the theorem remains valid. Therefore, Assumption 4.3 holds in most scenarios, and cases where it fails due to the non-continuity of  $E$  do not affect the correctness of Corollary 4.5 and Theorem 4.6.

## A.2 The Proof of Corollary 4.5.

Consider an object set  $R^* = \mathcal{V}^{(k)} \cap \mathcal{A}^{(k)}$ . From Theorem 4.4, we know that for each pair  $(v_i^{(k)}, \text{Asp}_i^{\text{Raw}(k)})$  within  $R^*$ , there exists a TIM,  $\hat{X}^{(k)}$  that captures their relationship. For simplicity, we denote the structured node of  $\text{Asp}_i^{\text{Raw}(k)}$  as  $a_i$ . Using  $\hat{X}^{(k)}$ , we model the

connectivity between any two nodes  $a_i$  and  $a_j$ , where the edge weight  $w_{a_i, a_j}$  represents their interaction in  $R^*$ . This is formulated as:

$$w_{a_i, a_j} = \text{Agg}[\text{Comb}(\hat{x}_{a_i, v_1}, \hat{x}_{a_j, v_1}), \text{Comb}(\hat{x}_{a_i, v_2}, \hat{x}_{a_j, v_2}), \dots, \text{Comb}(\hat{x}_{a_i, v_n}, \hat{x}_{a_j, v_n})] \tag{11}$$

where  $\text{Comb}(\cdot)$  is a function that combines  $\hat{x}_{a_i, v_k}$  and  $\hat{x}_{a_j, v_k}$  (e.g., multiplication), and  $\text{Agg}(\cdot)$  aggregates multiple Asp-Node-Asp relationships (e.g., averaging). When both  $\text{Agg}(\cdot)$  and  $\text{Comb}(\cdot)$  are consistent across datasets, Corollary 4.2 ensures that  $w_{a_i, a_j}$  remains a TIM, meaning the constructed graph structure is a TIM. The resulting graph  $G^{(\text{New})}$  consists solely of nodes from  $\mathcal{A}^{(k)}$ , capturing the physical property of "relation" among all objects in  $\mathcal{A}^{(k)}$ . Next, we employ a domain-invariant structural embedding method,  $\text{TopoEmb}(\cdot)$ , as follows:

$$X_{\text{Asp}}^{(k)} = \text{TopoEmb}(G^{(\text{New})}) \tag{12}$$

According to Corollary 4.2,  $X_{\text{Asp}}^{(k)}$  is a TIM. Therefore, in the  $\mathcal{D}_{\text{GG}}$ ,  $X_{\text{Asp}}^{(k)}$  is a TIM with respect to  $R^2 = \{\mathcal{A}^{(k)} \mid \text{DS}^{(k)} \in \mathcal{D}_{\text{GG}}\}$ .

## A.3 The Proof of Theorem 4.6.

For a graph dataset  $\text{DS}^{(k)} \in \mathcal{D}_{\text{GG}}$  containing  $\mathcal{V}$  and  $\mathcal{A}$ , according to Theorem 4.4 and Corollary 4.5, we have the attribute TIM of  $\mathcal{A}$   $X_{\text{Asp}}^{(k)}$  and the relation TIM  $\hat{X}^{(k)}$ , which captures the relationship between a node  $v_i$  and  $\text{Asp}_j^{\text{Raw}(k)}$ . Then, we have:

$$\begin{aligned}
X_{\text{FN}i}^{(k)} &= \text{Ext}_{\text{FN}}(\hat{X}_{i, \cdot}^{(k)}, X_{\text{Asp}}^{(k)}) \\
&= \text{Agg}(\text{Enc}(\hat{X}_{i, 1}^{(k)}, X_{\text{Asp} 1}^{(k)}), \text{Enc}(\hat{X}_{i, 2}^{(k)}, X_{\text{Asp} 2}^{(k)}), \\
&\quad \dots, \text{Enc}(\hat{X}_{i, d}^{(k)}, X_{\text{Asp} d^{(k)}}^{(k)}))
\end{aligned} \tag{13}$$

where  $\text{Agg}(\cdot)$  is an aggregation function (e.g., averaging), and  $\text{Enc}(\cdot)$  represents an encoding function. If both  $\text{Agg}(\cdot)$  and  $\text{Enc}(\cdot)$  are domain-invariant, then  $\text{Ext}_{\text{FN}}(\cdot)$  is also domain-invariant. According to Corollary 4.2,  $X_{\text{FN}}^{(k)}$  is a TIM with respect to  $R^3 = \{(v_i, \mathcal{A}^{(k)}) \mid v_i \in \text{DS}^{(k)}, \text{DS}^{(k)} \in \mathcal{D}_{\text{GG}}\}$  in  $\mathcal{D}_{\text{GG}}$ .

## A.4 Scalability and Complexity Analysis

Our method, which expands each feature into a node, may seem intuitively complex. However, this complexity is acceptable for large-scale datasets, and TIG can be easily applied to them. This is mainly due to two reasons: **1) Low feature dimensionality in large-scale datasets.** While large-scale graphs contain a high number of nodes ( $n$ ), the feature dimensionality ( $d$ ) is typically limited, i.e.,  $d \ll n$ . For example, the OGB-Product dataset has 2.44 million nodes but only 100-dimensional features. Thus, the preprocessing time for feature nodes remains negligible. **2) Optimization and acceleration strategies.** In large scale scenarios, We design several coding tricks in the implementation. For example, during the construction of  $\mathcal{G}^{(2)}$ , we design acceleration strategies such as randomized sampling to efficiently approximate inter-feature relationships. Furthermore, the augmented feature embeddings are propagated back to original nodes using batch processing and optimized message-passing schemes, which greatly improves efficiency.

**Complexity Analysis.** Compared to standard GNNs, TIG introduces additional complexity during the construction and processing

**Table 6: Hyper-parameters settings of TIG.**

| Scenarios             | Models          | Learning rate | Weight decay | #Epochs | Dim #1 | Dim #2 | Dim #3 | $\lambda$ |
|-----------------------|-----------------|---------------|--------------|---------|--------|--------|--------|-----------|
| Scenario (1)          | TIG             | 0.0001        | 1e-3         | 300     | 256    | 512    | 1024   | 0.15      |
| Scenarios (2) and (3) | TIG (1-shot)    | 0.0001        | 5e-4         | 200     | 16     | 128    | 256    | 0.1       |
|                       | TIG (3-shot)    | 0.0001        | 5e-4         | 200     | 16     | 16     | 256    | 0.1       |
|                       | TIG (5-shot)    | 0.0001        | 5e-4         | 200     | 32     | 128    | 256    | 0.1       |
| Scenario (4)          | TIG (Cora)      | 0.00001       | 5e-4         | 10      | 16     | 64     | 16     | 0.2       |
| Graph classification  | TIG             | 0.00001       | 5e-4         | 300     | 256    | 1024   | 1024   | 0.2       |
| In-domain             | TIG (Cora)      | 0.0001        | 1e-3         | 300     | 128    | 1024   | 1024   | 0.1       |
|                       | TIG (CiteSeer)  | 0.0001        | 1e-3         | 300     | 128    | 512    | 2048   | 0.1       |
|                       | TIG (PubMed)    | 0.0001        | 1e-3         | 300     | 128    | 1024   | 2048   | 0.1       |
|                       | TIG (Photo)     | 0.0001        | 1e-3         | 300     | 256    | 1024   | 2048   | 0.15      |
|                       | TIG (Computers) | 0.0001        | 1e-3         | 300     | 256    | 512    | 2048   | 0.15      |
|                       | TIG (CS)        | 0.0001        | 1e-3         | 200     | 64     | 512    | 1024   | 0.2       |
|                       | TIG (Physics)   | 0.0001        | 1e-3         | 200     | 256    | 1024   | 2048   | 0.15      |

**Table 7: Cross-domain graph classification. Our test settings followed SCORE [25]. Methods with “\*” are reported from SCORE.**

| Setting   | Method          | IMDB-B                             |                                    | COLLAB                             |                                    | COX2                               |                                    |
|-----------|-----------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|
|           |                 | Acc                                | F1                                 | Acc                                | F1                                 | Acc                                | F1                                 |
| One-shot  | GPPT* [18]      | 50.15 $\pm$ 0.75                   | 44.16 $\pm$ 6.70                   | 47.18 $\pm$ 5.93                   | 42.87 $\pm$ 7.70                   | <b>78.23 <math>\pm</math> 1.38</b> | 44.68 $\pm$ 1.17                   |
|           | AIO* [20]       | 60.07 $\pm$ 4.81                   | 56.88 $\pm$ 0.80                   | 51.66 $\pm$ 0.26                   | 47.78 $\pm$ 0.10                   | 76.14 $\pm$ 5.51                   | 49.62 $\pm$ 10.42                  |
|           | GPF-plus* [3]   | 57.93 $\pm$ 1.62                   | 55.55 $\pm$ 2.03                   | 47.24 $\pm$ 0.29                   | 41.24 $\pm$ 0.31                   | 33.78 $\pm$ 1.52                   | 30.90 $\pm$ 11.56                  |
| Zero-shot | ULTRA (3g)* [4] | 49.25 $\pm$ 0.00                   | 38.87 $\pm$ 0.00                   | 64.53 $\pm$ 0.00                   | 55.36 $\pm$ 0.00                   | 77.75 $\pm$ 0.00                   | 43.74 $\pm$ 0.00                   |
|           | SCORE* [25]     | 61.83 $\pm$ 1.60                   | 60.91 $\pm$ 2.18                   | 65.45 $\pm$ 1.05                   | 57.71 $\pm$ 1.82                   | <u>78.08 <math>\pm</math> 1.33</u> | 49.24 $\pm$ 3.55                   |
| Zero-shot | TIG (IMDB-B)    | -                                  | -                                  | <b>69.69 <math>\pm</math> 0.71</b> | <b>68.31 <math>\pm</math> 0.74</b> | 56.37 $\pm$ 3.99                   | <u>51.00 <math>\pm</math> 3.87</u> |
|           | TIG (COLLAB)    | <u>66.40 <math>\pm</math> 3.54</u> | <u>66.24 <math>\pm</math> 3.60</u> | -                                  | -                                  | 59.07 $\pm$ 4.82                   | <b>53.91 <math>\pm</math> 4.51</b> |
|           | TIG (COX2)      | <b>69.05 <math>\pm</math> 1.90</b> | <b>68.85 <math>\pm</math> 1.96</b> | <u>69.32 <math>\pm</math> 0.81</u> | <u>68.12 <math>\pm</math> 0.73</u> | -                                  | -                                  |

**Table 8: Performance on in-domain scenarios. Methods with “\*” are reported from [31].**

| Method           | Cora                               | CiteSeer                           | PubMed                             | Photo                              | Computers                          | CS                                 | Physics                            |
|------------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|
| Raw features*    | 57.90 $\pm$ 3.90                   | 57.60 $\pm$ 2.85                   | 79.55 $\pm$ 0.98                   | 80.99 $\pm$ 1.65                   | 75.59 $\pm$ 1.69                   | 89.92 $\pm$ 0.95                   | 93.18 $\pm$ 0.45                   |
| PCA*             | 56.36 $\pm$ 4.14                   | 48.29 $\pm$ 3.18                   | 82.80 $\pm$ 0.91                   | 74.92 $\pm$ 1.82                   | 71.26 $\pm$ 1.34                   | 87.12 $\pm$ 0.73                   | 92.48 $\pm$ 0.47                   |
| SVD              | 57.87 $\pm$ 4.58                   | 63.77 $\pm$ 2.28                   | 82.69 $\pm$ 0.96                   | 80.34 $\pm$ 3.81                   | 71.70 $\pm$ 1.67                   | 88.17 $\pm$ 0.22                   | 92.22 $\pm$ 0.77                   |
| GRACE*           | 83.20 $\pm$ 1.87                   | 70.99 $\pm$ 2.29                   | 85.46 $\pm$ 0.54                   | 91.93 $\pm$ 0.83                   | 85.36 $\pm$ 0.82                   | 91.84 $\pm$ 0.37                   | OOM                                |
| BGRL*            | 81.57 $\pm$ 2.07                   | 70.10 $\pm$ 2.04                   | 83.67 $\pm$ 0.84                   | 92.34 $\pm$ 0.73                   | 86.51 $\pm$ 1.53                   | 92.12 $\pm$ 0.63                   | 95.42 $\pm$ 0.41                   |
| GBT*             | 82.45 $\pm$ 1.97                   | 70.16 $\pm$ 2.32                   | 85.69 $\pm$ 1.22                   | 92.44 $\pm$ 0.48                   | 87.46 $\pm$ 0.82                   | 92.39 $\pm$ 0.78                   | 95.07 $\pm$ 0.23                   |
| DGI*             | 83.24 $\pm$ 2.12                   | 71.23 $\pm$ 2.37                   | 84.62 $\pm$ 0.83                   | 92.32 $\pm$ 0.49                   | 86.12 $\pm$ 0.73                   | 92.47 $\pm$ 0.60                   | 94.47 $\pm$ 0.50                   |
| FUG*             | <u>84.45 <math>\pm</math> 2.45</u> | 72.43 $\pm$ 2.92                   | 85.47 $\pm$ 1.13                   | 93.07 $\pm$ 0.82                   | 88.42 $\pm$ 0.98                   | 92.89 $\pm$ 0.45                   | 95.45 $\pm$ 0.27                   |
| SGRL             | 84.21 $\pm$ 2.50                   | <b>73.81 <math>\pm</math> 1.79</b> | <u>85.82 <math>\pm</math> 0.50</u> | 93.73 $\pm$ 1.45                   | <b>89.51 <math>\pm</math> 1.02</b> | 93.24 $\pm$ 0.76                   | <u>95.84 <math>\pm</math> 0.44</u> |
| TIG              | <b>85.51 <math>\pm</math> 2.09</b> | <u>73.47 <math>\pm</math> 0.78</u> | <b>86.88 <math>\pm</math> 0.61</b> | <b>93.99 <math>\pm</math> 0.87</b> | <u>89.01 <math>\pm</math> 0.70</u> | <b>93.49 <math>\pm</math> 0.63</b> | <b>95.86 <math>\pm</math> 0.47</b> |
| GCN (Supervised) | 82.80 $\pm$ 0.00                   | 72.00 $\pm$ 0.00                   | 84.80 $\pm$ 0.00                   | 92.42 $\pm$ 0.00                   | 86.51 $\pm$ 0.00                   | 93.03 $\pm$ 0.00                   | 95.65 $\pm$ 0.00                   |

of  $\mathcal{G}^{(1)}$  and  $\mathcal{G}^{(2)}$ . Specifically, constructing the graphs requires sampling  $s$  Asp-Node-Asp paths, which incur a cost of  $O(s)$ . We then perform SVD on  $\mathcal{G}^{(2)}$ , which has a complexity of  $O(d^3)$ , where  $d$  denotes the feature dimensionality. These steps are conducted entirely during the preprocessing stage. During training and downstream forward propagation, the model propagates feature embeddings to nodes and performs encoding, which has a computational complexity of  $O((d+1)n)$ . Thus, the overall additional complexity introduced by TIG can be summarized as:  $O(s + d^3)$  for preprocessing,  $O((d+1)n)$  for encoding. For large-scale datasets (where  $n \gg d$  and  $n \gg s$ ), the overhead from  $d$  and  $s$  is negligible, resulting in an overall complexity of approximately  $O(n)$ .

## B Experimental Settings

### B.1 Datasets

We follow existing studies [3, 31, 37] and select many widely used general graph datasets. These datasets are divided into four groups: (1) Citation datasets including Cora, CiteSeer, and PubMed; (2) Co-purchase datasets including Photo and Computers; (3) Co-author datasets including CS and Physics; (4) Molecular Graphs including BBBP, Tox21, ToxCast, ClinTox, MUV, HIV; And in appendix, we added: (5) Widely used graph classification datasets including IMDB-B, COLLAB, COX2, following [25].

## B.2 Compared Methods

In each scenario, we compared TIG with outstanding works, which include: (a) Universal graph pretraining models: OFA [9], GraphControl [35], All-in-One [19], FUG [31], GCOPE [30], ULTRA (3g) [4], SCORE [25]. (b) Graph self-supervised learning methods: GRACE [36], BGRL [21], GBT [1], DGI [23], and SGRL [5]. (c) Prompt-based methods: GPPT [18], GraphPrompt [11], GPF [3], and GPF-plus [3]. (d) Other meaningful baselines, including Raw features, PCA (node feature processed by PCA), SVD (node feature processed by SVD), GCN [7].

## B.3 Implementation Details

In scenarios (1), we followed the setup from previous work [31], where the model was first self-supervised on the training dataset. Then, a simple downstream L2 classifier was finetuned with the pretrained model fixed. We split the training, validation, and test sets in a 10%:10%:80% ratio and ran the experiment 20 times to report the mean and standard deviation. In scenarios (2) and (3), we also first self-supervised the TIG model, keeping it fixed in downstream settings. For each class,  $n$  samples (where  $n = 1$  in scenarios (2) and  $n = 3$  or  $n = 5$  in scenarios (3)) were randomly chosen and labeled. In scenario (2), for TIG, we used a single sample per class as the prototype vector without any prompts or finetuning, and predicted labels were determined based on the highest similarity between the node and prototype vectors. In scenario (3), we used 3 or 5 samples to finetune a downstream L2 classifier. In all few-shot scenarios, we run the experiments 500 times to report the average and standard deviation. In scenario (4), following GPF, 50 samples are used for fine-tuning. We employed a two-layer GCN as the graph encoder, a one-layer GCN as the TIM extractor, and PReLU as the activator.

All of our experiments were conducted on a server equipped with an Intel Core i5-12400 CPU, 32GB of RAM, an NVIDIA RTX 3090 graphics card, and 24GB of video memory.

**Hyper-Parameter Settings.** The hyper-parameters of TIG are summarized in Table 6. Here,  $\lambda$  is the weight for the positive part of the loss in the Scattering loss. Dim #1 represents the number of dimensions used for dimensional augmentation, Dim #2 represents the output dimensionality of the TIM extractor, and Dim #3 denotes the number of dimensions in the representations.

## C Supplementary Experiments.

We further validate the performance of TIG on graph-level tasks. Although universality across various tasks is not the primary scope

of this paper, we report TIG’s performance on graph classification tasks as shown in Table 7, as TIG is a graph foundation model. We selected three commonly used graph-level datasets: 1) IMDB-BINARY, which is a movie collaboration dataset [28], 2) COLLAB, a scientific collaboration dataset [28], and 3) COX2, a chemical dataset [8, 15]. It is worth noting that IMDB-BINARY and COLLAB do not provide raw node features; therefore, we generated their features via one-hot encoding of node degrees. For our experimental setup, similar to node classification, we first generate node representations using TIG and then obtain graph representations through pooling. During testing, we followed existing work [25] by utilizing class prototype vectors generated from the training set for zero-shot scenario. The hyper-parameter settings of TIG are in Table 6. The experimental results are presented in Table 7. It is worth noting that the methods we compare are state-of-the-art techniques specifically designed for graph classification, many of which incorporate additional optimization strategies tailored to the task. However, TIG does not include such specific optimizations. Even so, TIG still achieved the best and second-best results in five out of six settings, further validating the model’s strong generalization capabilities. This also demonstrates that focusing solely on extracting Transfer-Invariant Metadata can significantly enhance a graph foundational model’s performance.

We also provided the in-domain performance of TIG, as shown in Table 8. The settings are followed FUG [31]. TIG also demonstrates strong performance in in-domain scenarios.

## D Limitations

The limitations of this work primarily lie in two aspects. First, our study focuses on exploring transfer-invariant metadata within node features, without explicitly investigating the transfer invariance of graph structures. Therefore, our approach is generally effective on homophily graphs, but it may encounter challenges on heterophily graphs. To address this, one potential solution, as Discussion 3 in Section 4, is to leverage prompts to unify the differences in  $R$  between homophily and heterophily graphs. Alternatively, as suggested in Section 2, a topology selector could be incorporated as a complementary mechanism. We will explore this in the future. Second, although Assumption 4.3 holds in most cases, and our design of the TIM Extractor effectively mitigates errors introduced by discontinuities in  $E$ . In some extreme scenarios, the assumption’s constraints on  $\text{Asp}$  may no longer be valid. In such cases, it would be necessary to redesign the TIM Extractor to ensure proper adaptation and maintain reliable performance across diverse graph datasets.